

TASK 2: Stock Trading Platform

Java Programming — CodeAlpha Internship

Task Requirements

- Simulate a basic stock trading environment
- Use features for market data and buy/sell operations
- Allow users to track portfolio performance over time
- Use Object-Oriented Programming (OOP) for stocks, users, and transactions
- Optionally include file I/O for persistent portfolio data

OOP Design — Classes Used

- Stock — holds symbol, company name, current market price
- Holding — represents shares owned in user portfolio
- Transaction — records each BUY/SELL event with details
- StockTradingPlatform — main class with all logic and menu

Key Features Implemented

- Random market price simulation (fluctuation each menu cycle)
- Buy stocks with cash balance validation
- Sell stocks with quantity and ownership validation
- Portfolio view with real-time Profit & Loss calculation
- Full transaction history display

Complete Java Source Code

```
import java.util.ArrayList;
import java.util.Random;
import java.util.Scanner;

// Represents a Stock
class Stock {
    String symbol;
    String name;
    double price;

    Stock(String symbol, String name, double price) {
        this.symbol = symbol;
        this.name = name;
        this.price = price;
    }
}

// Represents a Portfolio holding
class Holding {
    String symbol;
    int quantity;
    double buyPrice;

    Holding(String symbol, int quantity, double buyPrice) {
```

```

        this.symbol    = symbol;
        this.quantity  = quantity;
        this.buyPrice  = buyPrice;
    }
}

// Represents a Transaction
class Transaction {
    String type;    // BUY or SELL
    String symbol;
    int    quantity;
    double price;

    Transaction(String type, String symbol, int qty, double price) {
        this.type    = type;
        this.symbol   = symbol;
        this.quantity = qty;
        this.price    = price;
    }
}

public class StockTradingPlatform {
    static ArrayList<Stock>      market      = new ArrayList<>();
    static ArrayList<Holding>    portfolio    = new ArrayList<>();
    static ArrayList<Transaction> transactions = new ArrayList<>();
    static double cashBalance = 10000.00;
    static Random random      = new Random();

    // Simulate market price fluctuation
    public static void simulateMarket() {
        for (Stock s : market) {
            double change = (random.nextDouble() - 0.48) * 5;
            s.price = Math.max(1.0, s.price + change);
            s.price = Math.round(s.price * 100.0) / 100.0;
        }
    }

    // Display available stocks
    public static void displayMarket() {
        System.out.println("\n--- Market Data ---");
        System.out.printf("%-8s %-20s %10s%n", "Symbol", "Name", "Price");
        System.out.println("-".repeat(40));
        for (Stock s : market)
            System.out.printf("%-8s %-20s $%9.2f%n", s.symbol, s.name, s.price);
    }

    // Buy a stock
    public static void buyStock(String symbol, int qty) {
        Stock found = null;
        for (Stock s : market) if (s.symbol.equals(symbol)) { found = s; break; }
        if (found == null) { System.out.println("Stock not found!"); return; }

        double total = found.price * qty;
        if (total > cashBalance) { System.out.println("Insufficient funds!"); return; }

        cashBalance -= total;
        // Check if already holding
    }
}

```

```

        for (Holding h : portfolio) {
            if (h.symbol.equals(symbol)) {
                h.quantity += qty;
                System.out.printf("Bought %d more %s at $%.2f%n", qty, symbol,
found.price);
                transactions.add(new Transaction("BUY", symbol, qty, found.price));
                return;
            }
        }
        portfolio.add(new Holding(symbol, qty, found.price));
        transactions.add(new Transaction("BUY", symbol, qty, found.price));
        System.out.printf("Bought %d shares of %s at $%.2f%n", qty, symbol, found.price);
    }

    // Sell a stock
    public static void sellStock(String symbol, int qty) {
        Holding target = null;
        for (Holding h : portfolio) if (h.symbol.equals(symbol)) { target = h; break; }
        if (target == null) { System.out.println("You don't own this stock!"); return; }
        if (target.quantity < qty) { System.out.println("Not enough shares!"); return; }

        Stock found = null;
        for (Stock s : market) if (s.symbol.equals(symbol)) { found = s; break; }

        double total = found.price * qty;
        cashBalance += total;
        target.quantity -= qty;
        if (target.quantity == 0) portfolio.remove(target);
        transactions.add(new Transaction("SELL", symbol, qty, found.price));
        System.out.printf("Sold %d shares of %s at $%.2f%n", qty, symbol, found.price);
    }

    // Display portfolio
    public static void displayPortfolio() {
        System.out.println("\n--- Your Portfolio ---");
        System.out.printf("Cash Balance: $%.2f%n", cashBalance);
        if (portfolio.isEmpty()) { System.out.println("No holdings."); return; }
        System.out.printf("%-8s %-6s %-12s %-12s %s%n",
            "Symbol", "Qty", "Buy Price", "Curr Price", "P&L");
        System.out.println("-".repeat(55));
        double totalPL = 0;
        for (Holding h : portfolio) {
            double curr = 0;
            for (Stock s : market) if (s.symbol.equals(h.symbol)) curr = s.price;
            double pl = (curr - h.buyPrice) * h.quantity;
            totalPL += pl;
            System.out.printf("%-8s %-6d $%-11.2f $%-11.2f $%.2f%n",
                h.symbol, h.quantity, h.buyPrice, curr, pl);
        }
        System.out.printf("Total P&L: $%.2f%n", totalPL);
    }

    // Display transaction history
    public static void displayTransactions() {
        System.out.println("\n--- Transaction History ---");
        for (Transaction t : transactions)
            System.out.printf("%s | %-6s | Qty: %-4d | Price: $%.2f%n",
                t.type, t.symbol, t.quantity, t.price);
    }

```

```

    }

    public static void main(String[] args) {
        // Initialize market
        market.add(new Stock("AAPL", "Apple Inc.", 175.50));
        market.add(new Stock("GOOGL", "Alphabet Inc.", 140.20));
        market.add(new Stock("TSLA", "Tesla Inc.", 210.80));
        market.add(new Stock("AMZN", "Amazon.com", 178.90));
        market.add(new Stock("MSFT", "Microsoft Corp.", 415.30));

        Scanner sc = new Scanner(System.in);
        boolean running = true;

        while (running) {
            simulateMarket();
            System.out.println("\n=== Stock Trading Platform ===");
            System.out.println("1. View Market    2. Buy Stock    3. Sell Stock");
            System.out.println("4. Portfolio        5. Transactions 6. Exit");
            System.out.print("Choice: ");
            int choice = sc.nextInt(); sc.nextLine();
            switch (choice) {
                case 1: displayMarket(); break;
                case 2:
                    displayMarket();
                    System.out.print("Symbol: "); String sym =
sc.nextLine().toUpperCase();
                    System.out.print("Qty: ");    int qty = sc.nextInt(); sc.nextLine();
                    buyStock(sym, qty); break;
                case 3:
                    System.out.print("Symbol: "); String s = sc.nextLine().toUpperCase();
                    System.out.print("Qty: ");    int q = sc.nextInt(); sc.nextLine();
                    sellStock(s, q); break;
                case 4: displayPortfolio(); break;
                case 5: displayTransactions(); break;
                case 6: running = false; break;
                default: System.out.println("Invalid choice!");
            }
        }
        sc.close();
        System.out.println("Goodbye!");
    }
}

```

Sample Output

```

=== Stock Trading Platform ===
1. View Market    2. Buy Stock    3. Sell Stock
4. Portfolio        5. Transactions 6. Exit

```

```

--- Market Data ---

```

Symbol	Name	Price
AAPL	Apple Inc.	\$ 176.23
GOOGL	Alphabet Inc.	\$ 138.95
TSLA	Tesla Inc.	\$ 213.40
AMZN	Amazon.com	\$ 180.10
MSFT	Microsoft Corp.	\$ 417.85